



Архітектура **С.Е.Н.:** Створення надійного локального агентного ядра на базі **Llama.cpp**, натхненого **Claude Code**

Цей дослідницький звіт представляє комплексний архітектурний план та глибокий аналіз для створення фреймворку С.Е.Н. (Core Engine Hub) — мінімалістичного, безпечного та кросплатформенного ядра для локальних агентів на основі великих мовних моделей (ВММ). Проект спрямований на реалізацію ідей, отриманих з аналізу витоку коду Claude, адаптованих для повноцінного локального використання. Ключовими принципами є виключна локальність, використання GGUF-моделей через `llama.cpp`, максимальна простота для користувача, але при цьому збереження потужності для вирішення завдань від базового кодингу до складних бізнес-процесів. Звіт охоплює всі аспекти розробки, включаючи архітектурний дизайн, механізми пам'яті, безпечну інтеграцію інструментів, технічні аспекти реалізації та стратегію забезпечення довготривалої стабільності.

Архітектурний дизайн: Послідовне масштабування від одиночного агента до рою

Розробка ефективного агентного фреймворку починається з фундаментального архітектурного вибору, який визначає його гнучкість, складність та здатність до масштабування. Для С.Е.Н. найбільш раціональною стратегією є послідовний розвиток, що передбачає спочатку побудову робастних можливостей одного агента, а потім додавання опціональних шарів оркестрації для багатьох агентів. Такий підхід дозволяє зберегти простоту користування на початковому етапі, коли головним завданням є створення надійного та потужного інструменту для вирішення широкого кола завдань, і одночасно запланувати подальший розвиток до більш складних, розподілених систем. Це

значно знижує початкову складність розробки та відповідає меті створення продукту, доступного для широкого кола користувачів, від домашніх до корпоративних .

Центральною ідеєю послідовної парадигми є створення двох типів агентів: "Planer" (планувальник) та "Executor" (виконавець) . У базовому режимі один агент виконує всі кроки завдання, від його розуміння до виконання інструментів. Коли користувач формулює надзвичайно складну задачу, фреймворк може автоматично або за допомогою команди користувача активувати режим рою . У цьому режимі верхній агент-оркеструвальник (планувальник) відповідає за декомпозицію загальної мети на менші, логічно пов'язані підзадачі, визначення залежностей між ними та координування роботи нижчих агентів-виконавців . Кожен виконавець є спеціалізованим агентом, оснащеним набором інструментів та знань, відповідних для виконання конкретного типу завдань, наприклад, "Research Agent", "Code Generation Agent" або "Debugging Agent" . Ця ієрархічна структура дозволяє ефективно розподіляти роботу та забезпечує гнучкість у вирішенні складних проблем.

Для ефективного планування та управління залежностями між завданнями в рамках рою необхідно відходити від простих послідовних ланцюгів і впроваджувати графові методи. Методи, такі як GAP (Graph-based Agent Planning), де під час планування будується граф залежностей, є більш потужними підходами для завдань з нелінійними залежностями, що часто зустрічаються в реальних сценаріях . Комунікація між агентами в таких системах повинна бути стандартизованою для забезпечення взаємодії між різними інструментами та фреймворками. Використання такого універсального адаптера, як Model Context Protocol (MCP), може спростити взаємодію між різними інструментами та фреймворками, забезпечуючи єдиний механізм обміну даними та запитами . Реалізація такого багатоагентного рою вимагає продуманого підходу до оркестрації. Фреймворк може починати з базового режиму роботи, де один агент виконує всі кроки завдання. Коли користувач формулює надзвичайно складну задачу, фреймворк може автоматично або за допомогою команди користувача активувати режим рою .

Ключовим елементом для створення справді продуктивної агентної системи є підтримка асинхронного виконання. Серійний підхід, де агент повинен чекати результату виконання одного інструменту, є головним обмеженням для довгих ланцюгів викликів і значно збільшує загальний час виконання складних, багатокрокових процесів . Тому фреймворк С.Е.Н. повинен використовувати асинхронне розгалуження, де планувальник може працювати над наступними

кроками, поки виконавці асинхронно виконують свої завдання . Це значно скорочує загальний час виконання складних, багатокрокових процесів і є ключовим елементом для створення справді продуктивної агентної системи. Для надзвичайно складних завдань, які вимагають координації великої кількості спеціалізованих агентів, необхідні продвинуті механізми, такі як відповідальність за Byzantine Fault Tolerance, які реалізовані в таких проектах, як Ruflo (раніше Claude Flow), хоча вони більше підходять для корпоративних сценаріїв . Для цілей даного дослідження, фокус на створенні гнучкої, але зручної системи, яка може поступово переходити від одноагентної до багатоагентної архітектури, є найбільш перспективним.

Архітектурний підхід	Переваги	Недоліки	Рекомендація
Уніфікована архітектура	Проста для розуміння та реалізації. Ефективна для широкого кола завдань.	Може бути занадто обмеженою для надзвичайно складних, розподілених задач. Не дозволяє використовувати сили спеціалізованих агентів.	Менш доцільний для даної мети, оскільки не враховує потенціал масштабування.
Послідовний розвиток (Один агент -> Рій)	Гнучкість та можливість масштабування. Збереження простоти користування на початку. Низька початкова складність.	Додаткова складність у реалізації шару оркестрації. Можливі проблеми з синхронізацією та комунікацією між агентами.	Рекомендований підхід. Найбільш практичний та адаптивний для досягнення поставленої мети.

Впровадження цієї послідовної парадигми вимагає створення чіткої структури класів та API. Головний клас **Agent** повинен мати внутрішню логіку для визначення складності завдання та вибору режиму роботи. При цьому важливо, щоб користувач мав можливість явно керувати цим процесом, наприклад, через специфічні команди або параметри запуску. Це дасть йому контроль над балансом між простотою та потужністю системи. Такий підхід дозволяє зберегти простоту користування, оскільки початковий досвід користувача не буде переповнений складними концепціями багатоагентних систем .

Система пам'яті та контексту: Персистентність, компакція та цілісність стану

Ефективна система пам'яті є критичним компонентом для створення агентної системи, здатної вирішувати завдання будь-якої складності та тривати довго . Натхнення, отримане з витоку коду Claude, підкреслює важливість

персистентної пам'яті, яка надає агенту індивідуальність, історичний контекст та довготермінові знання. Для досягнення цього в рамках локального фреймворку на базі `Llama.cpp` необхідно реалізувати багаторівневу систему пам'яті, яка поєднує довгострокову, короткотермінову та оперативну пам'ять. Ця система повинна бути спроектована таким чином, щоб забезпечити не лише збереження інформації, але й її ефективне управління для запобігання деградації продуктивності та пошкодженню стану.

Перший та найпростіший рівень — персистентна конфігурація, натхненена конвенцією `CLAUDE.md`. Це текстовий файл (наприклад, `.agent-config.md` або просто `agent.md`), який зберігає довгострокову інформацію про агента: його особливості, інструкції для моделі, історичні дані про успіхи та провали, а також довготермінові цілі. Це не просто набір параметрів, а частина системного промпта, яка формує індивідуальність агента. При кожному запуску фреймворк повинен читати цей файл і включати його вміст у системний промпт або в контекстне вікно, надаючи агенту постійний "особистий" контекст. Цей підхід уже став стандартом у популярних інструментах, таких як Cursor, OpenCode та Codex, що свідчить про його ефективність. Це дозволяє агенту пам'ятати про себе між сесіями, що є фундаментальною властивістю для довгострокового навчання та адаптації. В рамках С.Е.Н., цей файл, який можна назвати `agent.md`, буде єдиним джерелом правди для довготермінової пам'яті та конфігурації.

Другий рівень — короткотермінова пам'ять, яка відповідає за управління поточним контекстом протягом однієї сесії. Це найважливіша область для оптимізації, оскільки KV-кеши є одним з ключових обмежень при роботі з довгими контекстами, що призводить до виснаження пам'яті. Ефективне управління контекстом, а не просто прагнення зберегти максимум інформації, є ключовим. Тут фреймворк може використовувати багатостратегічні підходи до компакції, які були виявлені у витоку коду Claude. Пропонується почати з простих, але ефективних стратегій: 1. **Snip**: Відрізання найстарішої частини контексту, коли він перевищує заданий ліміт. 2. **Microcompact**: Стиснення відрізаної частини до короткого зведення, яке потім замінює її в контексті. Реалізація цих двох методів сама по собі дасть значний вииграш у довжині контексту. Для більш складних сценаріїв можна розробити прогресивну систему, яка включає інші стратегії, такі як **Context Collapse**, **Autocompact**, та **Reactive Compact**, які витік показав як існуючі у Claude. Ці методи можуть включати динамічне стиснення контексту на основі його релевантності, історії та поточної мети агента. Ключовим підходом до управління короткотерміновою пам'яттю є **Context Engineering**, який охоплює три основні аспекти: контекстне

управління, контекстну обробку та контекстне згенерування . Конкретні стратегії включають **Write** (створення контексту), **Select** (вибір лише найбільш релевантної інформації), **Compress** (стиснення довгих текстів) та **Isolate** (розподіл контексту на логічні блоки, щоб уникнути взаємного перешкоджання) . Особливо важливим є принцип ізоляції, який полягає у розділенні контексту на логічні блоки, наприклад, окремо для пам'яті, виходу інструментів та інструкцій до завдання, що запобігає перешкоджанню між різними типами інформації .

Третій, і найпотужніший, рівень — довготермінова пам'ять, яка виходить за межі однієї сесії. Для цього найчастіше рекомендується трьохрівнева система: контекстне вікно для оперативної пам'яті, база даних ключ-значення для середньотермінової структурованої інформації та векторна база даних для довготермінової семантичної пам'яті . Фреймворк повинен підтримувати інтеграцію з популярними локальними векторними базами даних, такими як FAISS або ChromaDB. Це дозволить агенту, використовуючи технологію збільшеної генерації на основі вибірки, отримувати доступ до знань, зібраних за багато сесій, що дозволяє йому вчитися на своєму досвіді та працювати з доменними знаннями, які не можна вмістити в контекстне вікно .

Однак, найбільш критичним аспектом є забезпечення цілісності стану. Аналогічно до того, як агент повинен мати пам'ять, він повинен мати і надійний механізм збереження цієї пам'яті. Слабким місцем у спрощеній архітектурі є використання простого JSON-файлу (`.agent_state.json`) для зберігання стану сесії . Лінійне додавання кроків призведе до файлу, який може досягати великих розмірів (50-200 MB) після довгої сесії, а збереження після кожного кроку без атомарних операцій може призвести до пошкодження даних при аварійному завершенні роботи . Для вирішення цього критичного недоліку необхідно замінити JSON-файл на **sqlite3**. SQLite є ідеальним вибором, оскільки він є вбудованим, сервером, не вимагає окремого процесу запуску, але при цьому забезпечує атомарні транзакції, що гарантує цілісність даних навіть після некоректного завершення роботи . Використання **sqlite3** дозволить організувати стан у вигляді таблиць (наприклад, `sessions`, `steps`, `context_chunks`), що полегшить масштабування, забезпечить надійність та дозволить легко виконувати запити для аналізу та відновлення сесій . Це фундаментальне покращення, яке перетворює спрощену архітектуру в надійний інструмент, здатний до довготривалої роботи.

Безпечна автономія: Інтеграція інструментів та система захисту

Створення автономної системи, здатної виконувати команди на локальному комп'ютері, несе за собою винятково важливу відповідальність за безпеку. Будь-яка помилка або зловмисне використання може призвести до серйозних наслідків, від видалення файлів до порушення конфіденційності даних ¹. Тому система дозволів та захисту є не просто додатковою функцією, а фундаментальною складовою архітектури фреймворку C.E.N. Натхнення з витоку коду Claude Code надало простий, але ефективний механізм для управління правами, який можна впровадити в будь-який локальний фреймворк. Ключовим завданням є створення екосистеми інструментів, яка є потужною, але водночас безпечним та надійною.

Центральною ідеєю є система дозволів з граційною деградацією. Агент починає свою роботу з високим рівнем автономії, що дозволяє йому ефективно виконувати завдання без зайвих інтеракцій. Однак, якщо агент допускає кілька послідовних помилок (наприклад, 3), система автоматично включає режим обмежень. У цьому режимі агент перестає виконувати дії самостійно і починає запитувати дозвіл на кожен крок. Це елегантне рішення для забезпечення безпеки, яке не впливає на продуктивність в нормальних умовах, але надає користувачеві повний контроль над ситуацією у разі проблем. Реалізація цього механізму вимагає введення лічильника помилок у стані агента. Коли лічильник досягає певного порогу, промпт, який надсилається моделі, динамічно змінюється, щоб вказати на необхідність отримання дозволу. Цей підхід є гарним прикладом мета-когнітивної логіки, де агент оцінює свою власну впевненість та здійснює відкладення, коли він є невпевненим. Цей механізм має бути описано в конфігураційному файлі `agent.md` у вигляді параметра `max_auto_errors`.

Другим рівнем захисту є контроль доступу до інструментів. Не кожен інструмент повинен бути доступний агенту за замовчуванням. Фреймворк повинен мати систему обмежень, яка чітко визначає, які інструменти може викликати агент на основі його ролі та поточного завдання. Це може бути реалізовано через схеми інструментів, які вже є хорошою практикою. Кожен плагін/інструмент повинен визначати свою схему, яка чітко описує його можливості та обмеження, наприклад, вказуючи, що вхідний файл має бути у форматі `.bench`. Це не лише поліпшує безпеку, але й допомагає моделі правильно використовувати інструменти. Більше того, для інструментів, які вимагають високих привілеїв

(наприклад, запис файлів, виконання команд терміналу), можна встановити правило, що їх перший виклик вимагає явного підтвердження користувача, навіть якщо інші правила дозволяють автоматичне виконання.

Третім, і найважливішим, рівнем захисту є захист від ін'єкцій та неконтрольованих дій. Оскільки агент може генерувати аргументи для інструментів (наприклад, команди для терміналу), це створює поверхню для атак. Необхідно впровадити механізми для виявлення та запобігання потенційним атакам. Наприклад, бібліотека `agentauth's PromptInjectionGuard` демонструє, як можна виявити ін'єкції в аргументах інструментів до того, як вони будуть передані функції. Для локальної системи це може означати ретельну перевірку та екранування аргументів командної строки перед їх виконанням. Також важливо використовувати принцип найменших привілеїв, запускаючи агента та його інструменти з обмеженими правами доступу до файлової системи та мережі.

Однак найкращим способом забезпечення безпеки є повноцінна пісочниця для виконання інструментів. Прості заходи, такі як `shell=False` у `subprocess`, недостатні для запобігання всім типам атак, оскільки агент все одно може імпортувати небезпечні модулі Python. Рекомендований підхід — використання повноцінної пісочниці, наприклад, через `Bubblewrap` (Linux) або `Docker`-контейнер з обмеженими правами. Це створює четвертий рівень захисту, як описано в чотиривимірній моделі безпеки для підсаджування агентів ШІ [1](#). Такий підхід дозволяє виконувати потенційно небезпечні дії в ізольованому середовищі з максимально обмеженими можливостями, що є найкращою практикою для домашнього та бізнес-середовища.

Технічна реалізація на базі **Llama.cpp**: Обмеження та практичні рекомендації

Вибір `Llama.cpp` як основи для створення локального агентного фреймворку є стратегічно вірним рішенням. Цей інструмент відзначається високою ефективністю та низькими вимогами до апаратних ресурсів, що дозволяє запускати потужні моделі на звичайних ноутбуках, що стало можливим завдяки прогресу у моделях, таких як `Qwen 3.6` [3](#). Однак, для успішної реалізації мети дослідження необхідно чітко розуміти технічні аспекти, обмеження та

архітектурні наслідки використання цього інструменту. `Llama.cpp` є не готовим агентним фреймворком, а ефективним інтерпретатором LLM, оптимізованим для локального виконання [3](#) [4](#). Він забезпечує швидкий інференс на CPU та GPU (включаючи Apple Silicon) [3](#). Отже, всі складні механізми, проаналізовані в цьому звіті — управління пам'яттю, оркестрація інструментів, система дозволів, компакція контексту — повинні бути реалізовані на рівні власного фреймворку, який використовує API `llama-cpp-python` для взаємодії з моделлю [5](#).

По-перше, необхідно враховувати апаратні обмеження. Хоча `Llama.cpp` дозволяє запускати великі моделі, вони все одно потребують значної кількості VRAM та RAM. Для надзвичайно складних завдань може знадобитися модель великих розмірів (наприклад, 70B+ токенів), яка може не вміститися в VRAM звичайного ноутбука. Фреймворк повинен бути осмисленим щодо цього. Одним з можливих підходів є адаптивне маршрутизування моделей, де для легких завдань використовується маленька, швидка модель, а для складних — велика, потужна модель, якщо вона доступна на даному обладнанні. Механізми, подібні до FrugalGPT, можуть бути впроваджені для того, щоб спочатку спробувати меншу модель, і лише при недостатній впевненості перейти до більшої. Це дозволить оптимізувати використання ресурсів та забезпечити належну продуктивність для завдань різної складності.

По-друге, локальна природа фреймворку має значення для його архітектури. Все, що потрібно для роботи — моделі, плагіни, конфігураційні файли та, можливо, локальні бази даних (наприклад, FAISS) — повинні працювати в рамках одного комп'ютера. Це виключає потребу у хмарних сервісах, що відповідає меті дослідження, але вимагає, щоб фреймворк був самодостатнім та не залежав від зовнішніх API для своєї основної функціональності. Це створює переваги у відносній простоті, оскільки немає необхідності управляти авторизацією, витратами на API та мережевими затримками, але також створює виклики у масштабуванні та розподіленій обробці.

Особливу увагу слід приділити інтеграції з `llama.cpp`. Відсутність нативного **tool calling** є ключовим обмеженням. `llama-cpp-python` не має нативної, надійної підтримки для структурованого виводу, подібного до OpenAI-стандарту. Модель може генерувати JSON у тексті, але без граматичного обмеження (`response_format`) ви отримаєте валідність ~60-80%. Найбільш надійним рішенням є використання `llama_cpp.Llama.create_completion` з `grammar` (описаною генеративною граматикою у форматі GBNF) або `response_format={"type": "json_object", "schema": ...}`. Це дозволяє

примусово формувати вивід LLM, але все одно обов'язково потрібна додаткова валідація вихідних даних через Pydantic перед виконанням.

Ще одне важливе обмеження стосується паралелізму. `asyncio.gather()` паралелізує виконання інструментів (I/O-операції), але сам LLM-крок залишається послідовним. На одній машині з 12-24 GB VRAM запустити 2+ інстанси моделі 14B+ практично неможливо без серйозного використання дискового простору, що різко збільшує затримку. Тому необхідно чітко документувати, що прискорення отримується лише на фазі `tool execution`, а не на фазі `LLM reasoning`. Для справжнього паралельного планування потрібен окремий `planner` на маленькій моделі (напр. 3B-7B), який делегує `executor` на великій моделі.

Нарешті, конфігурація та запуск мають значення для кросплатформенності. `llama.cpp` є кросплатформним, але деталі налаштування можуть відрізнятись. Наприклад, для використання AMD iGPU на Linux необхідно встановити Vulkan та збирати `llama.cpp` з прапорцем `-DGGML_VULKAN=1`, а також надати користувачеві права доступу до GPU через групи `render` та `video` [4](#). Для Apple Silicon нативна підтримка є однією з ключових переваг `llama.cpp` [3](#). Фреймворк C.E.H. повинен абстрагувати ці деталі, надаючи простий CLI-інтерфейс (`typer`), який дозволяє користувачеві запускати агента з вказанням шляху до GGUF-файлу та інших параметрів, а сам фреймворк турботиться про правильну конфігурацію `llama-cpp-python`.

Стратегія довготривалої стабільності: Обрання залежностей та архітектурних патернів

Щоб забезпечити роботу проекту C.E.H. протягом років, обрання залежностей є критично важливим. Принципи обрання повинні базуватися на мінімізації поверхні оновлень, використанні довготривалої підтримки та уникненні швидко змінюючихся AI-екосистем. Основні принципи для C.E.H. включають: максимальну використання стандартної бібліотеки Python, мінімум сторонніх залежностей (< 30 прямих пакетів), та жодних AI-фреймворків "все-в-одному", таких як LangChain чи LlamaIndex, які характеризуються нестабільністю та постійними змінами API [1](#). Такий підхід мінімізує ризики "dependency hell" та

гарантує, що ядро фреймворку залишиться стабільним навіть при зміні зовнішніх умов.

Рекомендований набір залежностей для довготривалої підтримки:

Пакет	Версія	Призначення	Чому обрано для довготривалої стабільності
llama-cpp-python	>=0.2.0, <0.4.0	Інференс GGUF-моделей	Офіційні біндинги до llama.cpp. API стабілізується, підтримує GGUF v3. Уникаймо 0.4+ до випуску LTS.
pydantic	>=2.5, <3.0	Валідація аргументів інструментів, конфігурації	Стандарт індустрії. v2 має чітку політику зворотної сумісності, підтримується до 2028+.
typer	>=0.9, <1.0	CLI-інтерфейс	Побудований на click, який не змінював мажорну версію з 2014. API стабільний, документація повна.
rich	>=13.7, <14.0	Кольоровий вивід, прогрес-бари, TUI	Використовується у pip, pytest, uv. Активна підтримка, назад сумісний.
structlog	>=23.0, <25.0	Структуроване логування (JSON)	Enterprise-стандарт для агентних систем. Працює поверх стандартного logging, не ламає API.
uv	>=0.4, <0.6	Менеджер віртуальних оточень та детерміноване встановлення	Заміна pip/poetry. Швидкий, детермінований, генерує uv.lock для ідеальної відтворюваності.

Цей набір залежностей є стабільним, активно підтримується та має чітку політику зворотної сумісності, що мінімізує ризики для довгострокового проекту . Використання uv разом з uv.lock є ключовим для забезпечення детермінованості: цей файл фіксує не лише версії прямих залежностей, а й всіх транзитивних, гарантуючи, що реплікація середовища через 2-3-5 років буде ідентичною .

Чого категорично уникати:

- `langchain`, `langchain-core`: API змінюється кожні ~4 місяці. Важкий, тисячі транзитивних залежностей .
- `llama-index`: Аналогічно. Фокус на RAG, не на агентній оркестрації. Швидко ламає зворотню сумісність .
- `crewai`, `autogen`: Експериментальні, орієнтовані на демо/хмару. Не підходять для локального production .
- `transformers`, `torch`: ~2-4 GB зайвих залежностей, постійні breaking changes. `llama.cpp` вже закриває інференс .
- `fastapi`, `uvicorn`: Не потрібні для локального CLI-агента. Додають мережевий шар, складність деплою та безпеки .

Стратегія підтримки на роки: 1. Фіксація **Python**: Використовувати LTS-версії, наприклад, 3.11 (до 2027) або 3.12 (до 2028). Уникати 3.13+ для production-

агентів до 2026. 2. Обмежене оновлення: Оновлювати залежності лише раз на квартал, тільки **patch/minor** версії. Перевіряти **changelog** та проходити тестування перед комітом. 3. Стабільність **GGUF**: Використовувати лише зрілі квантизаційні формати, такі як **Q4_K_M** або **Q5_K_M**, які підтримуються **llama.cpp** з 2023 року. 4. Логіка "недовіри": Валідація вхідних даних > 90% коду. Агентні системи ламаються не через бібліотеки, а через невалідовані аргументи інструментів.

Готовий **pyproject.toml**:

```
[project]
name = "c-e-h"
version = "0.1.0"
description = "C.E.H. (Core Engine Hub) - Minimalist local agent framework"
requires-python = ">=3.11"
dependencies = [
    "llama-cpp-python>=0.2.0,<0.4.0",
    "pydantic>=2.5,<3.0",
    "typer>=0.9,<1.0",
    "rich>=13.7,<14.0",
    "structlog>=23.0,<25.0",
]

[project.optional-dependencies]
dev = ["pytest>=7.0", "ruff>=0.4", "mypy>=1.8"]

[tool.uv]
no-build-isolation = false
compile-bytecode = true
```

Цей набір залежностей та стратегія підтримки є основою для створення інфраструктурного стеку, який витримає оновлення ОС, зміни версій Python та еволюцію локальних LLM без переписування ядра C.E.H. .

Reference

1. The Four Layers of Sandboxing LLM Agents: From Kernel ... - LinkedIn <https://www.linkedin.com/pulse/four-layers-sandboxing-llm-agents-from-kernel-abhay-bhargav-wpdwc>
2. Decoupling AI Coding Agents into Programmable, Embeddable ... <https://arxiv.org/html/2604.11045v1>
3. 有问题，就会有答案 - 知乎 <https://www.zhihu.com/en/article/1890410136155169799>
4. ~21 tok/s Gemma 4 on a Ryzen mini PC: llama.cpp, Vulkan, and the ... <https://dev.to/hrodrig/21-toks-gemma-4-on-a-ryzen-mini-pc-llamacpp-vulkan-and-the-messy-truth-about-local-chat-m82>
5. Artifact Analysis of Ollama, LM Studio, and llama.cpp - arXiv <https://arxiv.org/html/2603.23996v1>
6. Computational tools and data integration to accelerate vaccine ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC11925797/>